FreeCAD Python CUIコマンドによるパーツの色変更の実現可能性に関する専門家レポート

1. はじめに：FreeCADのPythonスクリプト環境の理解

FreeCADは、その統合されたPythonインタープリタを通じて高度にスクリプト化可能なプラットフォームとして設計されています。これにより、ユーザーはグラフィカルユーザーインターフェース（GUI）を直接操作することなく、Pythonコードを用いてFreeCADドキュメント内の様々な要素をプログラム的に作成、変更、および分析することが可能です[1]。このプログラムによる制御は、単純なジオメトリの作成にとどまらず、Pythonの汎用性を活用した複雑な操作にまで及びます。FreeCADの内部ツール、モジュール、さらにはワークベンチの大部分がPythonで実装されており、FreeCADエコシステムにおけるPythonスクリプトの深い統合と強力な能力を明確に示しています[1]。

本レポートにおける主要な問いは、「FreeCAD Python CUIコマンドでパーツの色を変えることが可能か？」というものであり、これはコマンドラインユーザーインターフェース（CUI）またはヘッドレス環境でFreeCADを実行しながら、視覚的プロパティ、特にパーツの色を変更する可能性に焦点を当てています。

初期回答の概要:
ユーザーの質問に直接答えると、FreeCAD Pythonコマンドを使用してパーツの色を変更することは可能です。しかし、その方法論と変更の即時的な視覚的影響は、FreeCADが完全なグラフィカルユーザーインターフェース（GUI）で動作しているか、ヘッドレス（CUI）環境で動作しているかによって大きく異なります。FreeCADが視覚的プロパティをどのように処理し、永続化するかに根本的な違いがあるため、このレポートではこれらの区別を詳しく説明し、両方のシナリオに対する実用的な解決策を提供します。

2. FreeCADの二重アーキテクチャ：AppオブジェクトとGuiオブジェクト

基本的な分離

FreeCADの設計思想は、オブジェクトのコアな幾何学的およびデータ定義と、画面上でのその視覚的表現との厳密な分離に基づいています。このアーキテクチャ上の分離は、主にAppとGuiという2つのPythonモジュールによって管理されています [2]。

Appモジュール（例：App.Document、App.DocumentObject、Part::Feature）は、オブジェクトの「何であるか」を定義する役割を担います。これには、そのジオメトリ、寸法、パラメトリックな関係、およびその他の基盤となるデータが含まれます。Appモジュールによって管理されるプロパティは「データプロパティ」と呼ばれます。これらのプロパティはオブジェクトの存在に不可欠であり、FreeCADがコンソールモード（CUI）またはGUIなしのライブラリとして実行されている場合でも完全にアクセス可能です [6]。これらはモデルの永続的で非視覚的な側面を表します。

対照的に、Guiモジュール（例：Gui.Document、Gui.ViewObject、ViewProvider）は、3Dビューでのオブジェクトの視覚的「どのように見えるか」を定義します。これには、色、線幅、表示モード（例：ワイヤーフレーム、シェーディング）、透明度などのプロパティが含まれます。これらは「ビュープロパティ」として知られており、グラフィカルユーザーインターフェースに本質的に結びついています。したがって、GuiオブジェクトおよびそれにGui関連するViewProviderは、FreeCAD GUIがアクティブにロードされ、実行されている場合にのみ利用可能でアクセス可能です [2]。

関連する情報として、「FreeCADではすべてがコアと表現に分離されている。これには3Dオブジェクトも含まれる。Appモジュールを介してオブジェクトの定義プロパティ（FreeCADではフィーチャと呼ばれる）にアクセスし、Guiモジュールを介して画面上での表示方法を変更できる」と述べられています [2]。また、「Appパート（ドキュメントオブジェクト）はオブジェクトのジオメトリを定義し、GUIパート（ビュープロバイダーオブジェクト）はオブジェクトが画面にどのように描画されるかを定義する」とも説明されています [3]。

プロパティスクリプトの含意

このAppとGuiの分離を深く理解することは、FreeCADのスクリプト作成、特に完全なGUIなしの環境でモデルを操作したり、モデルに影響を与えたりする場合に極めて重要です。

「データプロパティ」（Appレベル）と「ビュープロパティ」（Guiレベル）の区別は、異なるFreeCADの操作モードでどのオブジェクト属性にアクセスし、変更し、永続化できるかを決定します [6]。具体的には、「データプロパティは…FreeCADがコンソールモードで使用されている場合やライブラリとして使用されている場合でも、常に存在する。ビュープロパティは…グラフィカルユーザーインターフェース（GUI）がロードされている場合にのみアクセス可能であり、FreeCADをコンソールモードやヘッドレスライブラリとして使用している場合はアクセスできない」と明記されています [6]。この事実は、CUIでの色変更の実現可能性に直接影響を与えます。

CUIでの色操作における根本的なアーキテクチャ上の制約

ユーザーの主な関心事は「CUIコマンド」を使用したパーツの色変更であり、これはヘッドレスまたはコンソール環境を意味します。複数の情報源 [2] は、色、線幅などの視覚的プロパティ（「ビュープロパティ」と呼ばれる）およびそれらを管理するオブジェクト（

Guiモジュール、ViewProvider）が、GUIがロードされている場合にのみ利用可能であると一貫して明示的に述べています。特に、最も直接的な色プロパティであるShapeColorは、Guiプロパティであることが示されています [8]。

ヘッドレス環境では、Gui.activeDocument()はNoneを返します [2]。また、

ViewProviderオブジェクトは作成もアクセスもできません [10]。したがって、純粋なCUI/ヘッドレスFreeCADインスタンスでPythonコマンドを介して

ShapeColorやその他のViewプロパティに直接アクセスまたは変更しようとすると、AttributeErrorなどのエラーが発生し、本質的に失敗します。

このアーキテクチャ上の設計は、コアデータと視覚的表現の意図的な分離を反映しています。これは、バッチ処理、シミュレーション、サーバーサイドでのモデル生成など、視覚的レンダリングを必要としない非視覚的オペレーションには非常に有益ですが、ヘッドレスコンテキストでの直接的な視覚的操作には根本的な制限を課します。このことは、CUIで「色変更」を実現するには異なるアプローチが必要であることを意味します。すなわち、Appレベルで色データを定義し、それが視覚的環境が利用可能になったときにGuiによって解釈され、レンダリングされるという方法です。

Table 1: FreeCADオブジェクトプロパティカテゴリ

この表は、「データプロパティ」と「ビュープロパティ」の間の重要な区別を明確かつ簡潔にまとめたものです。この区別は、FreeCADにおける色操作、特にヘッドレス操作に関する説明の基礎を成します。これは、プロパティのアクセス可能性と永続性を支配する基盤となるメカニズムを理解するというユーザーの暗黙のニーズに直接対応し、CUI環境で特定の操作がなぜ可能であるか、あるいは不可能であるかを明確にします。

カテゴリ	モジュール	説明	アクセシビリティ	例	参照元
データプロパティ	App	オブジェクトの「物理的」または本質的な特性とジオメトリを定義します（例：寸法、トポロジカル形状、マテリアルプロパティ）。	ヘッドレス/コンソールモードを含む常にアクセス可能。ファイル保存時に永続化されます。	Length, Width, Height, Shape, App.Material プロパティ（DiffuseColor, AmbientColor）。	[6]
ビュープロパティ	Gui	3Dビューでのオブジェクトの視覚的表現を定義します（例：色、線幅、表示モード）。ViewObject または ViewProvider に結びついています。	GUIがロードされている場合にのみアクセス可能。ヘッドレス/コンソールモードではアクセス不可。デフォルトでは元に戻すスタックの一部ではありません。	ShapeColor, Line Color, Point Color, Display Mode, Transparency（ビューレベル）。	[6]

3. PythonによるFreeCADオブジェクトの外観管理（GUI有効時）

ShapeColorプロパティ

FreeCAD GUIがアクティブな場合、オブジェクトの全体的な色を変更する最も直接的な方法はShapeColorプロパティを使用することです。ShapeColorはオブジェクトのViewObjectのプロパティであり、Gui.activeDocument().getObject("ObjectName")を介してアクセスされます[8]。

色は、赤、緑、青、およびオプションでアルファ（透明度）の浮動小数点値のタプルとして表現され、各コンポーネントは0.0から1.0の範囲です（例：(R, G, B)または(R, G, B, A)）[8]。

コード例：**GUIモードでのShapeColorの設定**

Python

```python
import FreeCAD as App
import FreeCADGui as Gui
import Part

# ドキュメントとオブジェクトが存在することを確認
doc = App.newDocument("ColorTestDoc")
box = doc.addObject("Part::Box", "MyBox")
box.Shape = Part.makeBox(10, 10, 10)
doc.recompute()

# オブジェクトのGUI表現にアクセス
# これはFreeCADがGUIで実行されている場合にのみ機能
try:
    gui_doc = Gui.activeDocument()
    view_object = gui_doc.getObject("MyBox")

    # 現在の色の出力（例：デフォルトのグレーの場合は (0.8, 0.8, 0.8)）
    print(f"Current ShapeColor: {view_object.ShapeColor}")

    # シェイプの色を赤に設定（R=1.0, G=0.0, B=0.0）
    view_object.ShapeColor = (1.0, 0.0, 0.0)
    print(f"New ShapeColor: {view_object.ShapeColor}")
```

```
    # オプション：透明度の設定（例：50%透明の場合は0.5）
    # view_object.Transparency = 50 # 透明度は通常GUIプロパティでは0-100
    # ShapeColorタプルの場合、アルファは0.0-1.0
    view_object.ShapeColor = (1.0, 0.0, 0.0, 0.5) # 50%透明の赤

    Gui.updateGui() # 3Dビューを更新して変更を反映
    doc.recompute() # ジオメトリの変更や依存関係がある場合はドキュメントを再計算
    print("Shape color changed to red with 50% transparency.")

except AttributeError:
    FreeCAD.Console.PrintError("GUIがアクティブでないか、オブジェクトが見つかりません。ShapeColorはGUIプロパティです。\n")
```

ShapeColorへの変更は通常、.FCStdドキュメントに保存されますが [14]、ビュープロパティであるため、そのアクセス可能性と変更はGUIがロードされていることに厳密に結びついています [6]。

マテリアルプロパティ（**App.Material**）

FreeCADオブジェクトは、Appレベルで定義されたマテリアルプロパティを持つこともでき、これらはその視覚的外観に影響を与えます。これらは永続性とヘッドレス操作に関してより堅牢な方法を提供します。

App.Materialオブジェクトには、DiffuseColor、AmbientColor、SpecularColor、EmissiveColor、SectionColor、Shininess、Transparencyなどのグラフィカルプロパティが含まれます [12]。これらのプロパティはデータプロパティであり [6]、オブジェクトの基本的な定義の一部であるため、ヘッドレスモードでもアクセスおよび変更が可能です。オブジェクトの

ViewProvider（GUI部分）は、これらのApp.Materialプロパティを解釈して、3Dビューでオブジェクトをレンダリングします [3]。

関連する情報として、「App Materialには、Diffuse Color、Ambient Color、Specular Color、Emissive Color、Shininess、Transparencyなどのプロパティがある」と述べられています [12]。

コード例：**App.Material**プロパティの設定

Python

```python
import FreeCAD as App
import FreeCADGui as Gui
import Part

# ドキュメントとオブジェクトが存在することを確認
doc = App.newDocument("MaterialTestDoc")
box = doc.addObject("Part::Box", "MyMaterialBox")
box.Shape = Part.makeBox(10, 10, 10)
doc.recompute()

# 新しいApp.Materialオブジェクトを作成
material = App.Material()

# マテリアルプロパティを設定（これらはデータプロパティ）
material.DiffuseColor = (0.0, 0.0, 1.0) # 青
material.Transparency = 0.5 # 50%透明（0.0から1.0）
material.AmbientColor = (0.1, 0.1, 0.1) # わずかなアンビエントライト効果

# マテリアルをオブジェクトのViewObjectに割り当てる。
# この割り当て自体はビュープロパティだが、App.Materialデータを参照する。
try:
    view_object = Gui.activeDocument().getObject("MyMaterialBox")
    view_object.Material = material # App.Materialオブジェクトを割り当て
    Gui.updateGui()
    doc.recompute()
    print("MyMaterialBoxのマテリアルプロパティが設定されました。")
except AttributeError:
    FreeCAD.Console.PrintError("GUIがアクティブではありません。ViewObjectにマテリアルを直接割り当てることはできません。\n")

# マテリアルをオブジェクトと共に保存するには、Appオブジェクトのプロパティであるか、
# 保存されるビュープロパティによって参照される必要がある。
# Part::Featureの場合、View.MaterialプロパティはApp.Materialを参照する。
# マテリアルオブジェクト自体はAppレベルの構成要素である。
```

Part Designボディの場合、Shape Appearance（おそらくShapeColor）が期待通りに機能し

ないが、Ambient color（マテリアルプロパティ）は機能するという報告があります [15]。これは、特定の複雑なオブジェクトタイプでは、

App.Materialが外観を制御するためのより信頼性の高い方法である可能性を示唆しています。

App.Materialの永続的な色ソリューションとしての役割（**GUI**なし操作向け）

ユーザーはCUIを介した色変更、つまりヘッドレス操作を求めています。ShapeColorはビュープロパティであり、ヘッドレス環境ではアクセスできません。しかし、DiffuseColorのようなApp.Materialプロパティは明示的に色タイプとしてリストされており [12]、データプロパティです [6]。データプロパティはヘッドレスモードでアクセス可能であり、保存時に永続化されます [6]。

したがって、App.Materialオブジェクトを作成または変更し、その色プロパティを設定することで、ヘッドレス環境でFreeCADドキュメント内に目的の色情報を効果的に「保存」できます。このドキュメントが後でGUIで開かれた場合、ViewProviderはこれらのApp.Materialプロパティを解釈して表示します。

これは、ヘッドレス環境で実行する必要があるFreeCADスクリプトで永続的な色情報を定義するための主要な方法です。これにより、焦点が直接的な視覚的操作からデータ駆動型の外観定義へと移ります。

Table 2: 一般的な色プロパティ形式

この表は、FreeCADのPython APIで期待される色値の正確なデータ型と範囲に関する迅速で明確な参照を提供します。これは、開発者が試行錯誤することなく色変更を正しく実装し、スクリプトが構文的および意味的に正しいことを保証するために不可欠です。

プロパティタイプ	形式	範囲	例	参照元
ShapeColor (ビュープロパティ)	(R, G, B) または (R, G, B, A) の浮動小数点タプル	各コンポーネントで0.0から1.0	赤の場合 (1.0, 0.0, 0.0)、半透明の緑の場合 (0.0, 1.0, 0.0,	[8]

			0.5)	
App.Materialプロパティ (DiffuseColorなど) (データプロパティ)	(R, G, B) の浮動小数点タプル	各コンポーネントで0.0から1.0	青の場合 (0.0, 0.0, 1.0)	[12]
Transparency (ViewObject上のビュープロパティ)	浮動小数点 (一部のコンテキストでは整数だが、通常はViewObject.Transparencyで0.0-1.0または0-100)	0.0 (不透明) から1.0 (完全に透明) または 0から100	50%透明の場合 0.5	[12]

4. ヘッドレスFreeCAD（CUI）と外観プロパティの課題

FreeCADにおける「ヘッドレス」の意味

FreeCADを「ヘッドレス」モードで実行するとは、グラフィカルユーザーインターフェースなしで操作することを意味します。これは通常、FreeCADCmd実行可能ファイルを使用するか、-c（コンソール）引数付きでFreeCADを起動することによって行われます [10]。Pythonスクリプトでは、これは完全なGUI環境を初期化しないことと同義です。

import FreeCADGui as Guiは可能ですが、重要なGUIコンポーネントは利用できません。

GuiモジュールとViewProviderオブジェクトが利用できない理由

前述のとおり、ビュープロパティとViewProviderオブジェクト（視覚的表現を管理するもの）は、アクティブなGUIに本質的に結びついています [3]。FreeCADがヘッドレスで実行される場合、

GUIドキュメントは作成されず、結果として

Appドキュメント内のフィーチャに対応するViewProviderオブジェクトは存在しません [10]。

Gui.activeDocument()にアクセスしようとするとNoneが返されます [2]。

関連する情報として、「ヘッドレスモードではビュープロバイダーを作成またはアクセスすることはできない。FreeCADGuiをヘッドレスモードでロードすることは可能だが、GUIドキュメントは作成されないため、ビュープロバイダーも存在しない」と明記されています [10]。

ShapeColorおよびその他の表示プロパティへの直接的な影響

ShapeColorはビュープロパティであるため [8]、ヘッドレス環境で

obj.ViewObject.ShapeColorまたはGui.activeDocument().getObject("ObjectName").ShapeColorを設定しようとすると、エラー（例：ViewObjectまたはGui.activeDocument()がNoneであるためのAttributeError）が発生します。同様に、Line Color、Point Color、Display Mode、Transparencyなどの他の視覚的プロパティ（ViewObjectに直接設定される場合）もアクセスできません [6]。

コード例：ヘッドレスモードでの**ShapeColor**の無効性の実証

Python

```python
import FreeCAD as App
import FreeCADGui as Gui # インポートされていても、アクティブなGUI要素は持たない
import Part
import os # 一時ファイルのパス操作のため

# GUIなしで明示的にセットアップすることでヘッドレス環境をシミュレート
# これは、通常GUIをロードする可能性のあるスクリプトでヘッドレス動作をテストするために重要
try:
    Gui.setupWithoutGUI()
    FreeCAD.Console.PrintMessage("FreeCADGuiがGUIなしでセットアップされました。\n")
except Exception as e:
    FreeCAD.Console.PrintMessage(f"GUIなしでGUIをセットアップできませんでした: {e}。既にヘッドレス
```

```
と仮定します。\n")

doc = App.newDocument("HeadlessColorTest")
box = doc.addObject("Part::Box", "MyHeadlessBox")
box.Shape = Part.makeBox(10, 10, 10)
doc.recompute()

# Gui.activeDocument()へのアクセスを試みる - Noneになるはず
gui_doc = Gui.activeDocument()
if gui_doc is None:
    FreeCAD.Console.PrintMessage("Gui.activeDocument()はヘッドレスモードではNoneです（期待通り）。\n")
else:
    FreeCAD.Console.PrintMessage("Gui.activeDocument()は予期せずNoneではありません。GUIがアクティブかもしれません。\n")

# ShapeColorにアクセスして変更を試みる - 失敗するはず
try:
    # この行がエラーを発生させるはず
    view_object = Gui.activeDocument().getObject("MyHeadlessBox")
    view_object.ShapeColor = (1.0, 0.0, 0.0)
    FreeCAD.Console.PrintMessage("ShapeColorの設定を試みました（この行は到達しないはずです）。\n")
except AttributeError as e:
    FreeCAD.Console.PrintMessage(f"エラー: {e}。ヘッドレスモードではViewObjectまたはShapeColorにアクセスできません。\n")
except Exception as e:
    FreeCAD.Console.PrintMessage(f"予期せぬエラーが発生しました: {e}\n")

# ドキュメントを保存（このセッションからのShapeColorの変更は含まれない）
temp_file_path = os.path.join("/tmp", "headless_box.FCStd") # 一時パスを使用
doc.saveAs(temp_file_path)
FreeCAD.Console.PrintMessage(f"ドキュメントが保存されました。ShapeColorを介した色変更は適用されませんでした。パス: {temp_file_path}\n")
```

ヘッドレス操作は厳密な環境であり、単にウィンドウがないだけではない

ユーザーの質問は「CUI」環境を意味しており、これはFreeCADでは「ヘッドレス」と同義です。FreeCADCmdまたはFreeCAD -cがヘッドレス操作に使用されることが明示されています [10]。さらに重要なのは、

Gui.setupWithoutGUI() [10] を使用すると、

FreeCADGuiをロードしながらも、GUIドキュメントやビュープロバイダーを作成しないことが可能であるという点です。これは単にウィンドウが非表示になっているというだけでなく、基本的なGUIインフラストラクチャが存在しないことを意味します。

したがって、FreeCADGuiをインポートするPythonスクリプトがヘッドレス実行を意図している場合、明示的にGui.setupWithoutGUI()を呼び出すか、Gui.activeDocument()がNoneであることを認識しておく必要があります。

このことは、ヘッドレスモードが単にGUIが表示されていない状態ではなく、明確な操作状態であることを明確にします。スクリプトは、Viewオブジェクトとそのプロパティの不在を処理するように設計する必要があります。これにより、永続的な色データのようなAppレベルのソリューションの必要性が強調されます。

5. ヘッドレス環境における外観管理戦略

データプロパティとしての外観データの保存（**App.Material**またはカスタムプロパティ）

これは、ヘッドレススクリプトで色を定義し、後でFreeCAD GUIでドキュメントを開いたときにその色が表示されるようにするための推奨されるアプローチです。

- App.Materialの使用:
  App.Materialオブジェクトを作成し、そのDiffuseColor、AmbientColorなどのプロパティを設定します 12。オブジェクトのViewObjectのMaterialプロパティ（例：obj.ViewObject.Material）はビュープロパティであり、ヘッドレスモードで直接設定することはできませんが、App.Materialオブジェクト自体はデータ構成要素です。標準のPart::Featureやその他のAppオブジェクトは、App.Materialオブジェクトを参照するMaterialプロパティを持つことがよくあります。このデータプロパティはドキュメントと共に保存されます。ドキュメントがGUIモードでロードされ

ると、ViewProviderは割り当てられたApp.Materialのプロパティを自動的に解釈し、適用します。
コード例：ヘッドレスモードでの**App.Material**の設定（永続性のため）

```python
import FreeCAD as App
import Part
import os

# Appレベルの操作にはFreeCADGuiのインポートやsetupWithoutGUI()は不要
# このスクリプトはFreeCADCmdまたは'FreeCAD -c'で実行されるように設計されています

doc = App.newDocument("HeadlessMaterialDoc")
box = doc.addObject("Part::Box", "MyPersistentBox")
box.Shape = Part.makeBox(10, 10, 10)

# App.Materialオブジェクトを作成（これはAppレベルのデータオブジェクト）
material = App.Material()
material.DiffuseColor = (0.0, 1.0, 0.0) # 緑
material.Transparency = 0.3 # 30%透明

# Appオブジェクトにカスタムカラープロパティを追加
# これはデータプロパティであり、永続化され、ヘッドレスでアクセス可能
box.addProperty("App::PropertyColor", "DesiredColor", "Appearance", "表示用の色
").DesiredColor = (0.0, 1.0, 0.0)
box.addProperty("App::PropertyFloat", "DesiredTransparency", "Appearance", "表示用の透明
度").DesiredTransparency = 0.3

doc.recompute()
temp_file_path = os.path.join("/tmp", "headless_material_box.FCStd")
doc.saveAs(temp_file_path)
FreeCAD.Console.PrintMessage(f"ドキュメントがカスタムの色/透明度プロパティと共に保存されま
した。パス: {temp_file_path}\n")

# このドキュメントがGUIで開かれた場合、カスタムのViewProviderまたはマクロが
# 'DesiredColor'と'DesiredTransparency'を読み取り、実際の視覚的表現
# (ShapeColor, Transparency)に適用できる。
```

- カスタムプロパティの追加:
  App.Materialに加えて、ユーザーはApp::PropertyColorタイプまたはApp::PropertyFloatタイプ（透明度用）のカスタムデータプロパティを、自身のAppオブ

ジェクトに直接追加できます 4。これらのカスタムプロパティはドキュメントと共に保存されます 6。ドキュメントがGUI付きのFreeCADで開かれた場合、カスタムのViewProvider [3] またはマクロ [16] がこれらのデータプロパティを読み取り、ビュープロパティ（ShapeColor、Transparency）に適用できます。これには、GUI環境内での追加のスクリプト作成またはカスタムViewProviderの定義が必要です。関連する情報として、「動的プロパティは、ユーザーによって追加されたもの、およびPythonコードを介して追加されたものである」と述べられています [6]。

シーングラフ表現の生成（視覚出力用）

CUIのインタラクティブな意味で直接「色を変更する」わけではありませんが、FreeCADはヘッドレスモードでもオブジェクトのシーングラフ表現を生成できます [10]。

Gui.setupWithoutGUI()の後に利用可能なGui.subgraphFromObject(obj)関数は、Coin3Dシーングラフを作成します。このシーングラフは、Appオブジェクトのプロパティ（設定されている場合はApp.Materialなど）やデフォルトのレンダリングルールから派生した外観データを取り込むことができます。このシーングラフは、完全なFreeCAD GUIを表示することなく、画像レンダリング（例：PivyとCoin3Dの機能を使用）に利用できます。

関連する情報として、「ただし、オブジェクトのシーングラフ表現を作成することは可能である：import FreeCADGui as Gui from pivy import coin Gui.setupWithoutGUI() doc = App.newDocument() obj = doc.addObject("Part::Box","Box") doc.recompute() view = Gui.subgraphFromObject(obj)」と述べられています [10]。

コード例：ヘッドレスモードでのシーングラフ生成（概念的）

Python

```python
import FreeCAD as App
import FreeCADGui as Gui
import Part
from pivy import coin # シーングラフ操作/エクスポートに必要
import os
```

```
try:
    Gui.setupWithoutGUI()
    FreeCAD.Console.PrintMessage("シーングラフ生成のためにFreeCADGuiがGUIなしでセットアップされました。\n")
except Exception as e:
    FreeCAD.Console.PrintMessage(f"GUIなしでGUIをセットアップできませんでした: {e}。既にヘッドレスと仮定します。\n")

doc = App.newDocument("ScenegraphRenderDoc")
box = doc.addObject("Part::Box", "MyRenderBox")
box.Shape = Part.makeBox(10, 10, 10)

# カスタムプロパティを設定。これはAppレベルのデータであり、シーングラフが読み取れる。
box.addProperty("App::PropertyColor", "RenderColor", "Render", "ヘッドレスレンダリング用の色").RenderColor = (1.0, 0.5, 0.0) # オレンジ
doc.recompute()

# シーングラフを生成
# この関数は、オブジェクトの現在の状態（Appデータを含む）とデフォルトのレンダリングルールに基づいてCoin3Dグラフを作成する。
scenegraph = Gui.subgraphFromObject(box)
if scenegraph:
    FreeCAD.Console.PrintMessage("シーングラフが正常に生成されました。\n")
    # 例：シーングラフをファイルに保存（例：Inventor形式の.iv）
    # scenegraph.write("output_box.iv") # このメソッドは直接利用できないか、単純ではない可能性がある。
    # Pivyを使用してシーングラフを走査・操作したり、画像にレンダリングしたりできる。
    # 例（概念的）：
    # from pivy.coin import SoOffscreenRenderer
    # renderer = SoOffscreenRenderer(scenegraph)
    # image = renderer.render()
    # image.save("rendered_box.png") # 画像処理ライブラリが必要
else:
    FreeCAD.Console.PrintMessage("シーングラフの生成に失敗しました。\n")

temp_file_path = os.path.join("/tmp", "scenegraph_box.FCStd")
doc.saveAs(temp_file_path)
```

ヘッドレスモードにおけるデータ永続性と即時視覚フィードバック

ユーザーの質問は「CUIコマンド」に関するものです。分析により、ヘッドレスモードでは直接的な視覚的変更（ShapeColor）は不可能であることが示されています。しかし、App.MaterialやカスタムのApp.PropertyColorは色データを永続的に保存できます。これは、ヘッドレスモードでは色データを「定義」できるものの、その変更を即座に「見る」ことはできないことを意味します。視覚的なフィードバックは、ドキュメントがGUIで開かれた場合にのみ発生します。

Gui.subgraphFromObject()関数[10]は、ヘッドレスモードから視覚的表現（シーングラフ）を生成する方法を提供し、それを画像としてレンダリングすることができます。これは、ヘッドレスでのデータ操作と視覚出力の間のギャップを埋めるものです。このことは、ヘッドレス環境での色管理の実践的なワークフローを明確にします。それはインタラクティブな視覚的変更ではなく、将来のレンダリングに影響を与えるプログラムによるデータ定義に関するものです。シーングラフを生成する能力は、ヘッドレス環境でも視覚的な成果物を生成できることを意味し、これは自動化されたレンダリングパイプラインにとって重要です。

6. 実践的なPythonコード例（統合と解説）

例1: GUIモードでのパーツのShapeColor変更

この例は、ShapeColorビュープロパティの直接的な操作を示しています。FreeCADGuiがアクティブである必要があります。これは、セクション3.1で既に提供されたコード例です。

例2: パーツのApp.Materialプロパティ設定（永続性のため）

この例は、Appレベルで色情報を定義する方法を示しています。このデータは永続化され、ロード時にGUIによって解釈されます。GUI環境とヘッドレス環境の両方で実行できます。これは、セクション5.1で提供されたカスタムプロパティを使用したコード例です。

例3: 外観プロパティを持つドキュメントの保存

この例は、変更を行った後にドキュメントを保存する方法を示しています。App.MaterialやカスタムApp::PropertyColorに影響を与えるようなデータプロパティは保存されるのに対し、Viewプロパティ（永続的なAppバックアップなしで直接設定されたShapeColorなど）は保存されないか、GUIがアクティブな場合にのみ関連するという点が強調されます。FreeCADの標準ファイル形式は.FCStdです [14]。

Python

```python
import FreeCAD as App
import Part
import os

# 新しいドキュメントとオブジェクトを作成
doc = App.newDocument("SavedColorDoc")
box = doc.addObject("Part::Box", "MySavedBox")
box.Shape = Part.makeBox(10, 10, 10)

# 方法1: カスタムのApp.PropertyColorを使用（データプロパティ）
# この色は保存され、後でカスタムのViewProviderまたはマクロによって読み取られる可能性がある。
box.addProperty("App::PropertyColor", "StoredColor", "Appearance", "後で表示するための色
").StoredColor = (0.0, 0.5, 1.0) # シアン

# 方法2: カスタムのViewProviderがApp.Materialを読み取るように設定されている場合
# (これはより複雑で、カスタムのPython FeatureまたはViewProviderが必要)
# 標準のPart::Featureの場合、そのViewObject.MaterialプロパティはApp.Materialを指す。
# GUIでこれを設定すると永続化される。ヘッドレスの場合、App.Materialが
# ViewProviderが見つけられるようにAppオブジェクトにアタッチされていることを確認する必要がある。
# 標準オブジェクトの簡単な方法は、GUIでオブジェクトのViewObjectにApp.Materialを直接設定するか、
# App.Materialオブジェクト自体がドキュメントの一部であることを確認することである
# (例: 名前付きオブジェクトとして。ただし、これは単純な色の場合にはあまり一般的ではない)。

# 標準のPart::Featureの場合、'View'プロパティ（Materialを含む）は、GUIセッションで設定されていれば保存される。
# ヘッドレスの場合、App.Materialオブジェクト自体が暗黙的にリンクされていない場合は管理する必要がある。
```

```
# 最も堅牢なヘッドレスな方法は、AppオブジェクトにPropertyColorを直接追加することである。

doc.recompute()
file_path = os.path.join("/tmp", "colored_part.FCStd") # ファイルを保存するパスを定義
doc.saveAs(file_path)
FreeCAD.Console.PrintMessage(f"ドキュメントが保存されました。パス: {file_path}。格納された色プロパティが含まれます。\n")

# 確認するには、FreeCAD GUIで 'colored_part.FCStd' を開き、MySavedBoxのプロパティを確認する。
```

7. まとめと推奨事項

ユーザーの質問への直接的な回答

FreeCAD Pythonコマンドを使用してパーツの色を変更することは可能です。しかし、その方法と有効性は、FreeCADがグラフィカルユーザーインターフェース（GUI）で実行されているか、ヘッドレス（CUI）環境で実行されているかによって大きく異なります。ShapeColorのような視覚的プロパティの直接的な操作は、GUIがアクティブな場合にのみ可能です。ヘッドレスモードでは、これらのGuiモジュールプロパティは利用できません。

FreeCADパーツの色をスクリプト化するためのベストプラクティス

- インタラクティブなGUIセッションの場合:
  即時の視覚的フィードバックを得るには、
  Gui.activeDocument().getObject("ObjectName").ShapeColor = (R, G, B, A)を使用します。
- ヘッドレス（**CUI**）自動化と永続性の場合**:**
  - 主要な方法**:** 色情報をデータプロパティとして保存します。最も堅牢な方法は、カスタムのApp::PropertyColorプロパティをAppオブジェクトに追加することです。
  - あるいは、App.Materialオブジェクトとそのグラフィカルプロパティ（DiffuseColorなど）を活用します。オブジェクトのView.MaterialプロパティにApp.Materialを割り当て

るにはGUIセッションが必要ですが、App.Material自体の定義は永続化されるAppレベルの操作です。その後、カスタムのViewProviderまたはマクロがこのApp.Materialを読み取り、ドキュメントがGUIで開かれたときに適用できます。
    - これらのデータプロパティは.FCStdファイルと共に保存され、後でドキュメントが開かれたときにGUIによって解釈されます。
- ヘッドレス視覚出力（例：画像生成）の場合: Gui.setupWithoutGUI()に続いてGui.subgraphFromObject(obj)を使用してシーングラフを生成します。このシーングラフは、オブジェクトのAppレベルのデータ（App.Materialプロパティを含む）を反映し、GUIなしでレンダリングを可能にします。

データ永続性とクロス環境互換性に関する考慮事項

FreeCADドキュメントと共に保存され、異なるFreeCADセッションや環境（GUIとヘッドレス）間でアクセスする必要がある情報については、常にデータプロパティを優先してください。ビュープロパティは、主にGUI内での実行時表示とインタラクションのためのものです。GUIとヘッドレスの両方で使用するスクリプトを設計する際には、条件付きロジック（Guiアクセス用のtry-exceptブロックなど）または個別の関数を実装して、異なる環境を処理する必要があります。

ユーザーは「CUIコマンド」について具体的に質問しました。分析により、CUIでは直接的な視覚的変更（ShapeColor）は不可能であることが示されています。しかし、App.MaterialやカスタムのApp.PropertyColorはCUIで設定できます。これらのAppレベルのプロパティは、色の「データ」定義であり、視覚的な表示命令ではありません。視覚的な表示は、GUIが存在し、このAppデータを解釈する場合にのみ発生します。このことは、CUIコンテキストでの「色変更」に対するユーザーの理解を再定義します。それは、CUI内でリアルタイムで色変更を見ることではなく、後でGUIによってレンダリングされるように、色情報をプログラムによってモデルのデータ構造に埋め込むことに関するものです。これは、ヘッドレスFreeCADのユーザーのメンタルモデルにとって重要な区別です。

FreeCADファイルは「セキュリティ上の理由から、埋め込みコードを一切含まない。パラメトリックオブジェクトを作成するために書いたPythonコードはファイル内に保存されない」と述べられています [3]。このことは、色変更をスクリプト化することはできますが、そのロジック（Pythonコード自体）は

.FCStdファイル内に保存されないことを意味します。もし、カスタムのApp::PropertyColorを解釈するためにカスタムのViewProviderやマクロを使用する場合（ヘッドレス永続性のために推奨されるように）、そのViewProviderやマクロはファイルを開くマシン上で利用可能でなければ

なりません。色データ（例：(1.0, 0.5, 0.0)）は保存されますが、そのデータを視覚的表現に適用する方法をFreeCADに指示するコード（標準のApp.Material解釈でない場合）は保存されません。これは配布に複雑さを加えます。ユーザーがヘッドレススクリプトでカスタム色プロパティを定義し、デフォルトのFreeCAD GUIがそれらを視覚的に自動的に解釈しない場合、これらのカスタムデータプロパティを読み取り、ビュープロパティに適用するPythonコード（例：マクロまたはカスタムワークベンチとして）も配布する必要があります。このことは、App.Materialが永続的な色にとって最も簡単な方法であることが多いという考え方を強化します。なぜなら、その解釈はより標準化されているからです。

将来の展望

FreeCADの継続的な開発により、Appレベルでレンダリング固有のプロパティをより直接的に定義する方法が導入され、ヘッドレス機能がさらに強化される可能性があります。しかし、基本的なApp/Guiの分離は今後も維持される可能性が高いと予想されます。

引用文献

1. Python - FreeCAD Documentation, 7月 18, 2025にアクセス、https://wiki.freecad.org/Python
2. FreeCAD Scripting Basics, 7月 18, 2025にアクセス、https://wiki.freecad.org/FreeCAD_Scripting_Basics
3. FreeCAD-documentation/wiki/Scripted_objects.md at main - GitHub, 7月 18, 2025にアクセス、https://github.com/FreeCAD/FreeCAD-documentation/blob/main/wiki/Scripted_objects.md
4. App FeaturePython - FreeCAD Documentation, 7月 18, 2025にアクセス、https://wiki.freecad.org/App_FeaturePython
5. Scripted objects - FreeCAD Documentation, 7月 18, 2025にアクセス、https://wiki.freecad.org/Scripted_objects
6. Property editor - FreeCAD Documentation, 7月 18, 2025にアクセス、https://wiki.freecad.org/Property_editor/en
7. Property editor - FreeCAD Documentation, 7月 18, 2025にアクセス、https://wiki.freecad.org/Property_editor/id
8. FreeCAD-documentation/wiki/Code_snippets.md at main - GitHub, 7月 18, 2025にアクセス、https://github.com/FreeCAD/FreeCAD-documentation/blob/main/wiki/Code_snippets.md
9. Code snippets - FreeCAD Documentation, 7月 18, 2025にアクセス、https://wiki.freecad.org/code_snippets
10. FreeCAD-documentation/wiki/Headless_FreeCAD.md at main - GitHub, 7月 18,

2025にアクセス、
https://github.com/FreeCAD/FreeCAD-documentation/blob/main/wiki/Headless_FreeCAD.md
11. Headless FreeCAD, 7月 18, 2025にアクセス、
https://wiki.freecad.org/Headless_FreeCAD
12. Material - FreeCAD Documentation, 7月 18, 2025にアクセス、
https://wiki.freecad.org/Material
13. Part Feature - FreeCAD Documentation, 7月 18, 2025にアクセス、
https://wiki.freecad.org/Part_Feature
14. Week 2 - Ganadev Prajapathy - Fab Academy Archive, 7月 18, 2025にアクセス、
http://archive.fabacademy.org/fabacademy2017/fablabtrivandrum/students/359/week2.html
15. [Problem] Part Design "Shape appearance" (color) doesn't change (Regression) #14779, 7月 18, 2025にアクセス、
https://github.com/FreeCAD/FreeCAD/issues/14779
16. Macro Colorize - FreeCAD Documentation, 7月 18, 2025にアクセス、
https://wiki.freecad.org/Macro_Colorize/en
17. Start up and Configuration - FreeCAD Documentation, 7月 18, 2025にアクセス、
https://wiki.freecad.org/Start_up_and_Configuration